Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92410133020**

Aim: To understand the process of convolution over the image and apply over the classification problem

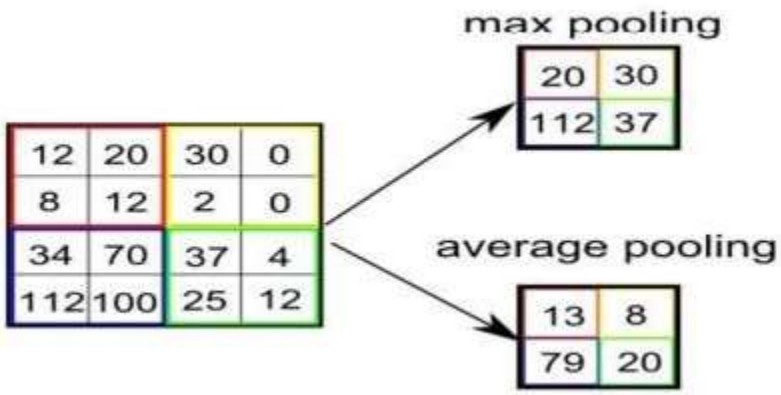
IDE: Google Colab

Theory:

Convolutional Neural Networks (CNN) are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy There are three types of layers in a convolutional neural network: i. Convolutional layer ii. Pooling layer iii. Fully connected layer Each of these layers has different parameters that can be optimized and performs a different task on the input data.

What is Pooling Layer?

Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data through dimensionality reduction. There are two types of Pooling i. Average Pooling. ii. Max Pooling Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.



What is Convolutional Layer?

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image. A convolutional layer contains a set of filters whose parameters need to be learned. The height and weight of the filters are smaller than those of the input volume. Each filter is convolved with the input volume to compute an activation map made of neurons.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92410133020**

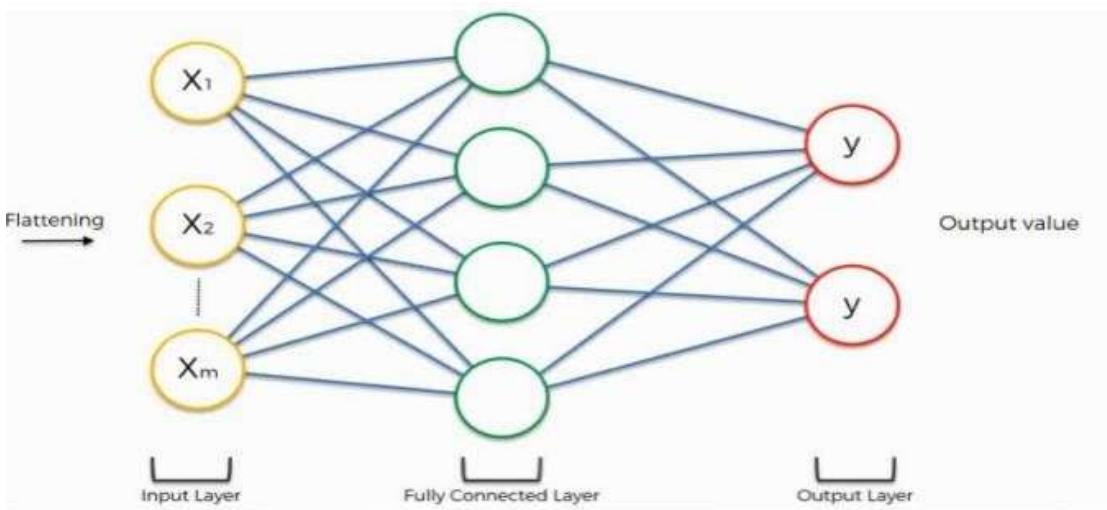
What is Fully Connected Layer?

A fully connected layer that takes the output of convolution/pooling and predicts the best label to describe the image We have three layers in the full connection step i. Input layer ii. Fully-connected layer iii. Output layer

Input Layer: It takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage.

Fully Connected Layer: It takes the inputs from the feature analysis and applies weights to predict the correct label.

Output Layer: It gives the final probabilities for each label.



ReLU Layer: ReLU is an activation function. Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable. The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data

7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

Program (Code):

To be attached with

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
import numpy as np

(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

train_images, test_images = train_images/255.0, test_images/255.0

class_names = ['airplane','automobile','bird','cat','deer','dog','frog','horse','ship','truck']

model1 = models.Sequential()
model1.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)))
model1.add(layers.MaxPooling2D((2,2)))
model1.add(layers.Conv2D(64,(3,3),activation='relu'))
model1.add(layers.MaxPooling2D((2,2)))
model1.add(layers.Conv2D(64,(3,3),activation='relu'))
model1.add(layers.Flatten())
model1.add(layers.Dense(64,activation='relu'))
model1.add(layers.Dense(10))

model1.compile(optimizer='adam',
          loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
          metrics=['accuracy'])

history1 = model1.fit(train_images, train_labels, epochs=10,
                validation_data=(test_images, test_labels))

model2 = models.Sequential()
model2.add(layers.Conv2D(32,(3,3),activation='relu',input_shape=(32,32,3)))
model2.add(layers.MaxPooling2D((2,2)))
```

```
model2.add(layers.Conv2D(64,(3,3),activation='relu'))
model2.add(layers.MaxPooling2D((2,2)))
model2.add(layers.Conv2D(64,(3,3),activation='relu'))
model2.add(layers.Flatten())
model2.add(layers.Dense(64,activation='relu'))
model2.add(layers.Dropout(0.5))
model2.add(layers.Dense(10))

model2.compile(optimizer='adam',
            loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
            metrics=['accuracy'])

history2 = model2.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))

plt.plot(history1.history['accuracy'], label='train_before')
plt.plot(history1.history['val_accuracy'], label='val_before')
plt.plot(history2.history['accuracy'], label='train_after')
plt.plot(history2.history['val_accuracy'], label='val_after')
plt.legend()
plt.title("Accuracy Comparison")
plt.show()

plt.plot(history1.history['loss'], label='train_before')
plt.plot(history1.history['val_loss'], label='val_before')
plt.plot(history2.history['loss'], label='train_after')
plt.plot(history2.history['val_loss'], label='val_after')
plt.legend()
plt.title("Loss Comparison")
plt.show()

predictions = model2.predict(test_images)
pred_labels = np.argmax(predictions, axis=1)

plt.figure(figsize=(10,5))
count = 0
for i in range(20):
    if pred_labels[i] == test_labels[i] and count < 5:
        plt.subplot(1,5,count+1)
        plt.imshow(test_images[i])
        plt.title("C:"+class_names[pred_labels[i]])
        plt.axis('off')
        count += 1
```

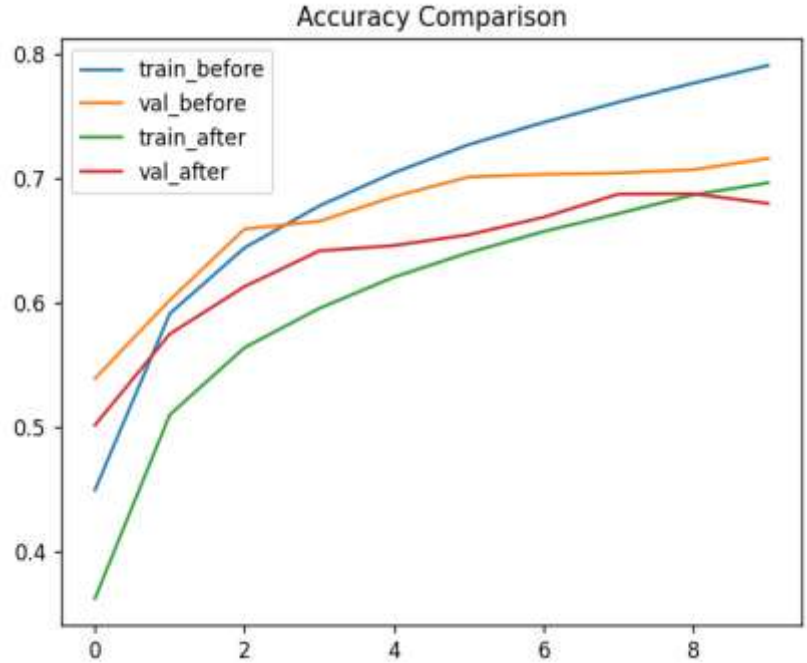
```
plt.show()

plt.figure(figsize=(10,5))
count = 0
for i in range(50):
    if pred_labels[i] != test_labels[i] and count < 5:
        plt.subplot(1,5,count+1)
        plt.imshow(test_images[i])
        plt.title("P:"+class_names[pred_labels[i]]+"\nA:"+class_names[test_labels[i][0]])
        plt.axis('off')
        count += 1
plt.show()
```

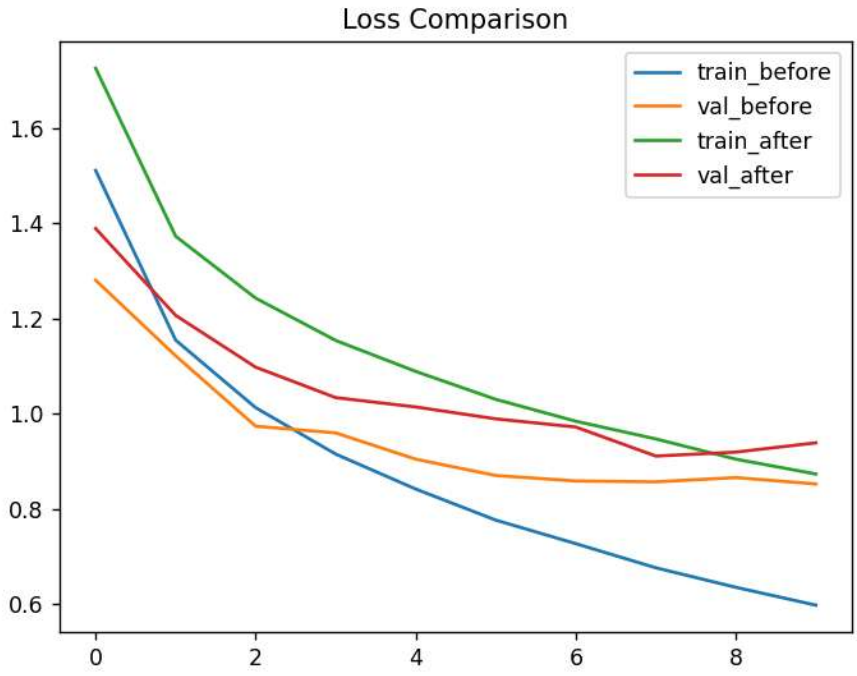
Results:

To be attached with

a. Training dataset
b. Model summary
c. Training and validation accuracy w.r.t epochs before regularization
d. Training and validation loss w.r.t epochs before regularization
e. Training and validation accuracy w.r.t epochs after regularization
f. Training and validation loss w.r.t epochs after regularization
g. Original v/s predicted labels for correct predicted observations
h. Original v/s predicted labels for incorrect predicted observations



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92410133020**



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**
Experiment No: 4	**Date:** / **Enrolment No:92410133020**

```
C:\Users\karna\OneDrive\Documents\6 th sem AI\LAB>python cnn_model.py
2026-03-25 23:01:25.807685: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to flo
ating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-03-25 23:01:34.898395: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to flo
ating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
C:\Users\karna\AppData\Local\Programs\Python\Python313\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an 'input
_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-03-25 23:01:50.397736: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in pe
rformance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 33s 18ms/step - accuracy: 0.4497 - loss: 1.5118 - val_accuracy: 0.5394 - val_loss: 1.2806
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 45s 29ms/step - accuracy: 0.5915 - loss: 1.1547 - val_accuracy: 0.6824 - val_loss: 1.1219
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 89s 33ms/step - accuracy: 0.6445 - loss: 1.0129 - val_accuracy: 0.6596 - val_loss: 0.9748
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 92s 39ms/step - accuracy: 0.6782 - loss: 0.9154 - val_accuracy: 0.6658 - val_loss: 0.9599
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 43s 27ms/step - accuracy: 0.7050 - loss: 0.8421 - val_accuracy: 0.6857 - val_loss: 0.9048
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 67s 42ms/step - accuracy: 0.7275 - loss: 0.7768 - val_accuracy: 0.7015 - val_loss: 0.8705
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 55s 35ms/step - accuracy: 0.7454 - loss: 0.7274 - val_accuracy: 0.7036 - val_loss: 0.8598
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 94s 43ms/step - accuracy: 0.7616 - loss: 0.6767 - val_accuracy: 0.7046 - val_loss: 0.8572
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 54s 35ms/step - accuracy: 0.7769 - loss: 0.6357 - val_accuracy: 0.7073 - val_loss: 0.8661
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 52s 33ms/step - accuracy: 0.7911 - loss: 0.5983 - val_accuracy: 0.7163 - val_loss: 0.8527
Epoch 1/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 58s 35ms/step - accuracy: 0.3625 - loss: 1.7256 - val_accuracy: 0.5019 - val_loss: 1.3890
Epoch 2/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 42s 27ms/step - accuracy: 0.5102 - loss: 1.3728 - val_accuracy: 0.5752 - val_loss: 1.2067
Epoch 3/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 47s 30ms/step - accuracy: 0.5643 - loss: 1.2429 - val_accuracy: 0.6133 - val_loss: 1.0979
```

```
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 47s 30ms/step - accuracy: 0.5643 - loss: 1.2429 - val_accuracy: 0.6133 - val_loss: 1.0979
Epoch 4/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 44s 28ms/step - accuracy: 0.5958 - loss: 1.1541 - val_accuracy: 0.6420 - val_loss: 1.0340
Epoch 5/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 55s 35ms/step - accuracy: 0.6210 - loss: 1.0890 - val_accuracy: 0.6463 - val_loss: 1.0144
Epoch 6/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 46s 30ms/step - accuracy: 0.6407 - loss: 1.0304 - val_accuracy: 0.6550 - val_loss: 0.9894
Epoch 7/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 83s 30ms/step - accuracy: 0.6576 - loss: 0.9845 - val_accuracy: 0.6691 - val_loss: 0.9723
Epoch 8/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 78s 27ms/step - accuracy: 0.6721 - loss: 0.9473 - val_accuracy: 0.6877 - val_loss: 0.9113
Epoch 9/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 85s 29ms/step - accuracy: 0.6871 - loss: 0.9048 - val_accuracy: 0.6879 - val_loss: 0.9197
Epoch 10/10
1563/1563 ━━━━━━━━━━━━━━━━━━━━ 33s 21ms/step - accuracy: 0.6968 - loss: 0.8736 - val_accuracy: 0.6801 - val_loss: 0.9391
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92410133020**

Pre Lab Exercise:

a. What is Convolution process? Explain giving example.

Convolution is the process of applying a filter (kernel) over an image to extract features like edges, textures.

b. What are different layers in CNN model?
   - Convolutional layer
   - Pooling layer
   - Fully connected layer

c. What is the requirement of the pooling layer in the CNN model?
   - Reduces size
   - Decreases computation
   - Removes noise

d. What is the requirement of the use of ReLU activation function after convolution step?
   - Removes negative values
   - Introduces non-linearity
   - Speeds up training

Observation and Result Analysis:

e. Nature of the dataset

Dataset contains labeled images of multiple classes (CIFAR-10). Images are RGB and categorized.

f. Training Process without regularization
   - Training accuracy increases
   - Validation accuracy may fluctuate
   - Overfitting may occur

g. After regularization in the training Process
   - Overfitting reduced
   - Better generalization
   - Stable accuracy

h. Observation over the Learning Curves
   - Accuracy increases with epochs
   - Loss decreases gradually
   - Gap between training & validation indicates overfitting

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To understand the process of convolution over the image and apply over the classification problem**	
Experiment No: 4	**Date:**	**Enrolment No:92410133020**

Post Lab Exercise:

a. Why CNN is preferred over ANN for images

CNN is better for images because it captures spatial features automatically.

b. Can CNN be applied over Text data? If yes, then how. If no, then why?

Yes, CNN can be used for text by converting words into vectors (embeddings).

c. What is the role of dropout layer?

- Prevents overfitting
- Randomly disables neurons during training

d. What will happen if maxpooling is replaced with minpooling?

- Important features may be lost
- Model performance decreases